

At times, there arises the need to carry out a static kernel configuration. This is primarily because, occasionally, the requisite functionality is so tied and embedded within the kernel that it becomes virtually impossible to make it dynamically loadable. Besides, certain security environments do not facilitate the loading and unloading of kernel modules and instead call for the requisite functionality to be compiled into the kernel in a static manner. In many ways, developing a custom kernel can more or less be considered as a mandatory and routine course of action for all the seasoned and professional BSD users out there. Though the process is a touch time consuming, it packs the promise of rendering multiple advantages to the FreeBSD environment and system in general. In contrast to the generic kernel that is expected to support a broad spectrum of devices, a custom kernel can be optimised and configured to offer support for only the hardware of a certain system that is under consideration. In the process, the user gets to avail of the following benefits:

- **Enhanced Boot Time:** In the case of a custom kernel, since it concerns itself with just the hardware of the computer, the time taken by the system to launch reduces drastically.
- **Lower Memory Consumption:** In most cases, the memory consumed by a custom kernel is less than the memory taken up by a generic kernel as it omits device drivers and other unutilised features. This is vital for the kernel code to stay inside the physical memory on all occasions, keeping the memory from being claimed by other applications in the process. Hence, with systems, that have a small RAM, to begin with, using a custom kernel proves to be extremely beneficial.
- **Extra Hardware Support:** Using a custom kernel gives users the liberty of including support for all devices that are not a part of the generic kernel.

However, before you proceed with configuring a custom kernel, you must always be mindful of the reason for doing so. Should there be a requirement for particular hardware support, there are chances that it may already be present as a pre-existing module.

Choosing the correct hardware for the system

Prior to making any edits to the kernel configuration file, users are advised to carry out an inventory check of the system's hardware. In the case of a dual-boot system, the details of the inventory can be generated from the second operating system. For instance, all relevant information revolving